Hierarchical Device–Edge–Cloud Task Offloading: An Experimental Study of DQN, Double DQN, Feedback, and Cloud Scheduling

Tasnim Mizaoui, Farah Cherif, Farah Cherif, and Doua Bousnina
INSAT, Tunis, Tunisia

***Abstract*—This paper presents an auditable device–edge–cloud offloading system and a documented experimental study of heuristic, centralized DQN, hierarchical DQN, and hierarchical Double DQN policies under delayed feedback. The canonical SimPy backend separates device placement, edge forwarding, and shared-cloud scheduling, while the learning integration uses per-decision observations and task or next-decision credit rather than a complete sequential multi-agent Gymnasium environment. The primary campaign replays 579 chronologically held-out tasks derived from Alibaba Cluster Trace 2017 after independent-task filtering; 2,891 accepted tasks are split into 1,734 training, 578 validation, and 579 test tasks and mapped to 24 devices, 3 edges, and one shared cloud worker. On the matched reference-topology comparison, centralized DQN has the highest mean deadline success (77.41%), Greedy/EDF the lowest mean P95 latency (12.95 s), and Greedy/FIFO and Greedy/EDF the lowest mean device energy (0.211 J/task). Hierarchical Double DQN reduces P95 and energy relative to hierarchical DQN, but has slightly lower success. Supporting feedback and topology studies expose further reversals, including a roughly 70% traffic reduction at an event threshold with a latency cost. The findings are descriptive and scenario-specific.**

***Index Terms*—mobile edge computing, task offloading, hierarchical reinforcement learning, discrete-event simulation, Double-DQN, reproducibility**

I. Introduction

Multi-access edge computing (MEC) places compute close to data producers while retaining access to larger cloud or HPC resources [1]. A task must therefore be mapped to a device CPU, an associated edge CPU, or a shared cloud queue. The choice couples execution time, queueing, wireless and backhaul transmission, device energy, and a relative deadline. A centralized controller can observe more state, but it also requires broader information sharing and can become a coordination bottleneck.

This project investigates a hierarchical alternative in which each tier makes only the decision exposed by its local contract. The device chooses local execution or its assigned edge; the edge chooses edge execution or cloud forwarding; and the cloud selects a queue discipline. The repository's contribution is primarily an auditable integration: a single SimPy backend executes all policies, with common task, timing, energy, and metric definitions. Double-DQN adapters and delayed adjacent-tier feedback are implemented at this boundary, but their scientific validation remains incomplete.

The article makes three bounded contributions. First, it specifies the three-tier model and policy interfaces. Second, it reports the committed three-seed synthetic baseline evidence without confusing it with trace measurements. Third, it identifies the exact gap between the implemented terminal one-step learning formulation and a sequential multi-agent RL environment. The remainder presents related work, the model, implementation, evaluation evidence, and limitations.

II. Related Work

ETSI defines MEC as a distributed service environment close to the access network [1]. Offloading literature consequently studies coupled placement, queueing, wireless transmission, energy, and deadline decisions; the communication–energy trade-off is surveyed in [2]. This paper narrows the decision boundary deliberately: device, edge, and cloud controllers receive different local observations and expose small discrete action spaces rather than solving one global optimization problem.

Simulation frameworks make different abstraction choices. EdgeCloudSim provides configurable edge-network and compute models [3]; iFogSim models resource management in IoT/fog workflows [4]; and PureEdgeSim targets cloud, edge, and mist scenarios with broad extensibility [5]. The present implementation uses a compact SimPy model so that policy injection and the three decision boundaries remain explicit. It therefore offers a controlled experimental instrument, not a replacement for the richer mobility, application, and network models of those frameworks.

Deep Q-learning established value-based control from replayed transitions [6], while Double DQN separates action selection and target evaluation to reduce overestimation [7]. Hierarchical and multi-agent MEC studies generally decompose placement or resource allocation across levels; federated reinforcement learning is a related future direction because it can distribute training without centralizing raw local experience. Hierarchical deep RL for joint caching and offloading similarly separates decisions across levels [8]; federated deep RL aggregates policies learned from distributed environments [9]; and centralized-training/decentralized-execution methods expose the information asymmetry that motivates local policy boundaries [10]. These approaches often assume sequential environments, richer multi-agent interaction, or explicit model aggregation. The present campaign is more modest: it compares DQN and Double DQN adapters under matched simulator

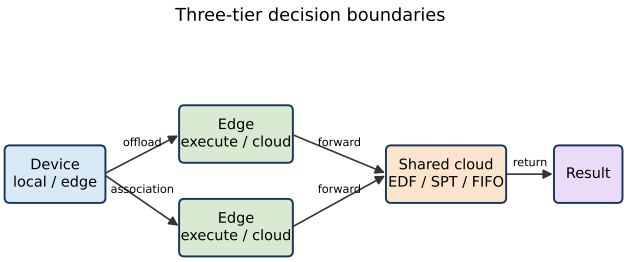


Fig. 1. Implemented device–edge–cloud decision boundaries.

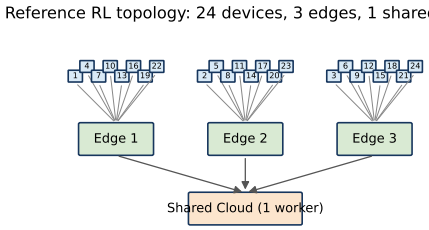


Fig. 2. Reference RL topology: 24 devices, three associated edges, and one shared cloud worker.

conditions and labels feedback and topology effects without claiming a universal hierarchy ranking.

Alibaba Cluster Trace 2017 supplies production datacenter arrival and compute records [11]. It is not a mobile-MEC measurement. The adapter derives computational work from completed instances, while payloads, deadlines, device assignment, battery, and wireless links are declared simulation assumptions.

III. Architecture and Proposed Approach

A. Three-tier architecture

The system has device, edge, and cloud decision levels. A device executes locally or sends a task to its fixed associated edge. The edge executes the task or forwards it to a shared cloud scheduler. The cloud chooses a queue discipline when a worker becomes available. The small simulator demonstration uses three devices and two edges; the RL campaign instead uses 24 devices, three edges, and one shared cloud. Device-to-edge associations are explicit, not learned as arbitrary routing choices. The campaign association is round-robin across the three edges, as shown in Fig. 2. The decision boundaries are shown in Fig. 1.

B. Task lifecycle and resource model

The lifecycle is arrival, device decision, optional device–edge upload, edge decision, optional edge–cloud upload, cloud dispatch, execution, and result return. Queues contribute waiting time, and every modeled link contributes serialization, propagation, and resource queue delay. Fig. 3 illustrates these paths.

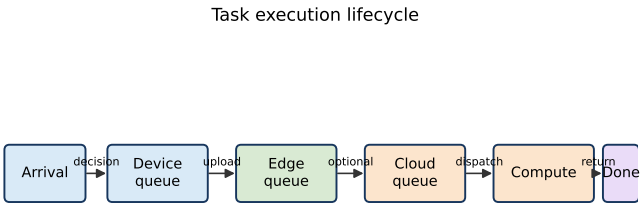


Fig. 3. Task lifecycle through queues, links, execution, and result return.

Each task has arrival time, decimal-MB input and result sizes, required MI, device identifier, relative deadline, and priority. For demand w MI and capacity c MIPS,

$$t_{\text{cpu}} = w/c. \tag{1}$$

For payload s decimal MB, bandwidth B Mbps, propagation p, and queue wait q,

$$t_{\text{link}} = 8s/B + p + q. \tag{2}$$

Latency is completion minus arrival, and a task meets its deadline when $t_{\text{completion}} \leq t_{\text{arrival}} + D$. Devices and edges use capacity-one SimPy resources. The current cloud has one worker and chooses EDF, shortest processing time (SPT), or FIFO. Device energy includes CPU energy per MI and transmit/receive power times radio duration; idle, reception-infrastructure, retransmission, and cloud energy are not modeled.

C. Observations and actions

The device observation contains normalized work, input size, deadline slack, battery fraction, device capacity, device–edge link state, and an optional delayed edge-congestion signal. Its actions are local execution and edge offloading. The edge observes task demand and deadline, edge queue/utilization, edge capacity, edge–cloud link state, and an optional delayed cloud signal. Its actions are edge execution and cloud forwarding. The cloud observes queue length, deadline summaries, estimated service time, and oldest wait; its actions are EDF, SPT, and FIFO. A centralized comparator uses a 15-feature global observation and selects device, edge, or cloud, with fixed EDF cloud dispatch. Lower-tier hierarchical agents do not receive raw global state.

D. Learning and reward

The DQN and Double DQN agents use two hidden layers of 64 units, replay capacity 8,000, batch size 32, learning rate 3×10^{-4}, target updates every 500 optimization steps, and Smooth L1 loss. Double DQN selects the next action with the online network and evaluates it with the target network; DQN takes the target-network maximum.

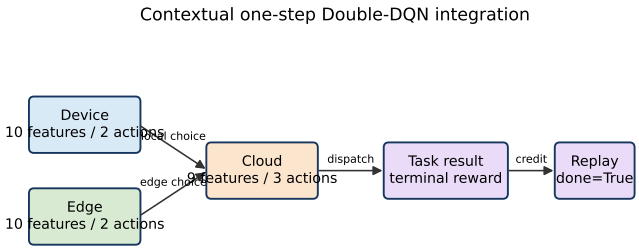


Fig. 4. Local observations and terminal or next-decision credit.

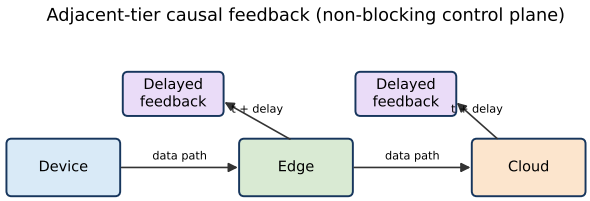


Fig. 5. Causal adjacent-tier feedback with a non-blocking control plane.

For non-cloud decisions, the shared terminal reward is

$$R = -\left(w_l \min\left(1, \frac{\log(1+L/D)}{\log(10001)}\right) + w_e \min\{3, E/E_0\} + w_d \mathbb{1}[\text{miss}] + w_c \min\{3, F/F_0\} \right), \tag{3}$$

with $(w_l, w_e, w_d, w_c) = (0.35, 0.25, 0.35, 0.05)$. Here L, D, E, and F denote end-to-end latency in seconds, deadline in seconds, device energy in joules, and feedback bytes. The implementation uses $E_0 = 0.3$ J and $F_0 = 64$ bytes from the default `RewardWeights` configuration; these are normalization references, not measured campaign means. The miss indicator is one when a deadline-feasible task misses its deadline. The cloud uses a queue-waiting-cost reward for contested dispatches, so its objective is not identical to the shared task reward.

The implementation is not a complete sequential multi-agent Gymnasium environment. Device and edge decisions receive terminal per-task credit. The RL campaign's cloud controller additionally uses next-decision credit with $\gamma = 0.9$ across contested dispatch decisions; singleton cloud queues are omitted because all three rules select the same job. Fig. 4 summarizes the distinction.

E. Feedback

Feedback is adjacent-tier and causal. A snapshot sampled at time t is available only after modeled propagation and serialization, at a later decision. Conditions are none, periodic, frequent, and event-triggered. The event trigger sends an initial snapshot, sends when the signal change reaches a threshold, and refreshes after five observations. Feedback bytes, modeled delay, and information age are recorded, but messages do not contend with data-plane links. Fig. 5 shows this control path.

IV. Implementation and Performance Evaluation

A. Software and reproducibility

The implementation targets Python 3.11 and uses SimPy, NumPy, Pandas, Matplotlib, PyYAML, PyTorch, and Gymnasium-compatible policy boundaries. The simulator/domain layers remain independent of Gymnasium and Stable-Baselines3. The final matched package verifies 52,110 per-task rows and 90 aggregate replay rows, task IDs and arrivals, configuration hashes, workload/topology fingerprints, checkpoint metadata, and disjoint seed sets. The package and commands are documented in `docs/experiments/final-matched/README.md`; raw traces and large checkpoints remain outside Git.

B. Workload and protocol

The Alibaba 2017 adapter accepted 2,891 independent tasks: 1,734 training, 578 validation, and 579 chronologically held-out test tasks. Arrival time is derived from `batch_task.create_timestamp`. Computational work is estimated from the mean work of valid completed instances using runtime, actual `real_cpu_avg`, and a configurable reference MIPS-per-core conversion; it is not a directly measured MI field. Multi-task jobs and incomplete/invalid instances are rejected, and DAG dependencies are not reconstructed.

The final package uses the best-tier-calibrated deadline rule with a one-second floor and validation-calibrated arrival compression 0.12793575549650835. Payload sizes, device assignment, device battery and radio state, wireless bandwidth and propagation, edge/cloud capacities, and relative deadlines are simulator assumptions. Accepted tasks are assigned round-robin to 24 devices and their configured edge associations. Alibaba is a production-cluster trace adapted to MEC, not a measured mobile-MEC workload.

The primary comparison uses 579 tasks on the reference topology: 24 devices, three edges, and one cloud worker. Random, Greedy/FIFO, and Greedy/EDF are fixed baselines evaluated with noise seeds 10001–10005. Centralized DQN uses the five standard-DQN checkpoints from training seeds 42–46, observes global state, and uses fixed EDF cloud dispatch. Hierarchical DQN and hierarchical Double DQN use the ten frequent-feedback bundles at the same five training seeds; their cloud rules are learned. For learned policies, five noise runs are averaged within each training seed, then the mean and sample standard deviation are computed across five training-seed means. Baseline deviations are across the five noise seeds. P95 is computed per replay before aggregation, so these two SDs describe different variation sources.

C. Matched six-policy comparison

Table I is the primary result. Device energy is reported in joules per task using the package's

TABLE I
MATCHED REFERENCE-TOPOLOGY COMPARISON. MEANS ± SD; P95 IS PER REPLAY BEFORE AGGREGATION.

Policy	Deadline success (%)	P95 latency (s)	Device energy (J/task)
Random	49.57 ± 0.95	137.96 ± 16.46	10.480 ± 0.720
Greedy/FIFO	75.79 ± 0.78	13.62 ± 0.49	0.211 ± 0.001
Greedy/EDF	75.89 ± 0.86	12.95 ± 0.43	0.211 ± 0.001
Centralized DQN	77.41 ± 1.37	15.71 ± 4.44	0.242 ± 0.067
Hierarchical DQN	74.63 ± 0.79	16.91 ± 3.32	0.260 ± 0.104
Hierarchical Double DQN	74.33 ± 1.66	15.08 ± 2.85	0.244 ± 0.065

TABLE II
ADDITIONAL METRICS FROM THE MATCHED PACKAGE. MEANS ± THE PACKAGE'S CORRESPONDING SD; BYTES ARE PER 579-TASK REPLAY.

Policy	Mean L (s)	Data B	Feedback B
Random	22.002 ± 2.681	$8.07 \pm 0.57\,10^8$	0
G/FIFO	2.619 ± 0.046	$12.86 \pm 0.12\,10^8$	0
G/EDF	2.472 ± 0.088	$12.86 \pm 0.12\,10^8$	0
C-DQN	2.794 ± 0.447	$12.54 \pm 0.68\,10^8$	0
H-DQN	3.045 ± 0.292	$12.72 \pm 1.08\,10^8$	$35.4 \pm 0.7\,10^3$
H-DDQN	3.015 ± 0.470	$12.04 \pm 0.42\,10^8$	$35.1 \pm 0.6\,10^3$

`mean_device_energy_j` metric; the denominator and aggregation follow the package summary. Values are descriptive means, not significance tests.

Figure 6 visualizes the same documented summary. Centralized DQN has the highest mean success, while Greedy/EDF has the lowest mean P95. Greedy/FIFO and Greedy/EDF share the lowest mean energy. Both hierarchical learners improve on Random for all three headline metrics, but neither beats Greedy/EDF on success, P95, or energy. Double DQN lowers mean P95 and energy relative to hierarchical DQN, with a small success decrease. These are not claims of statistical significance.

D. Supporting feedback, topology, and cloud analyses

The older DQN/Double-DQN feedback study is separate from the matched six-policy comparison. It found reference-topology success means of 75.01%, 74.28%, and 74.63% for DQN under none, periodic, and frequent feedback, versus 73.39%, 73.49%, and 74.33% for Double DQN. Double DQN had lower mean P95 for none and frequent feedback and lower energy in all three modes, but paired bootstrap intervals included zero. Held-out topology variants changed these relationships. At event threshold 0.5, Double DQN used 10,430 feedback bytes per run versus 35,126 for frequent feedback, about 70% less, with higher P95. The control messages have modeled bytes, delay, and information age, but do not contend with data-plane links.

The cloud-rule ablation is also separate. With Double-DQN upstream weights on the reference topology, learned, EDF, SPT, and FIFO achieved 74.33/15.08, 74.24/15.29, 74.82/12.36, and 73.90/17.82 for success (%)/P95 (s). Fixed SPT was best on the reported aggregate metrics. Each checkpoint selected one rule whenever at least two jobs waited, indicating constant selection behavior under these contested-queue evaluations.

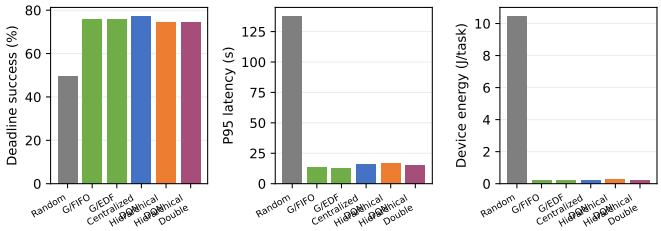


Fig. 6. Documented means for the six matched policies; lower is better for P95 and energy.

E. Reporting limitations

The matched package reports mean latency, P95 latency, deadline success, device energy, task communication bytes, and feedback bytes; Table II adds the metrics not included in Table I. Device energy is the simulator's `mean_device_energy_j`: total modeled device CPU and radio energy divided by the tasks in a replay, then aggregated according to the package's policy-specific seed procedure. It excludes idle, reception-infrastructure, retransmission, and cloud energy. The package's task communication bytes are data-plane payload bytes; feedback bytes are control-plane bytes. Feedback currently does not contend with data-plane links, so its communication cost should not be interpreted as queueing interference.

Centralized DQN observes global state and therefore has an information advantage over the hierarchical policies; centralized state-collection traffic is not included in the communication accounting. The five evaluation seeds vary simulated bandwidth and service-time noise on one chronological test window, not five independent trace periods, and the five training seeds provide limited evidence. The reported standard deviations are descriptive and reflect different aggregation sources for fixed and learned policies.

F. Preliminary synthetic baseline

The earlier three-seed, ten-task development smoke experiment remains a simulator sanity check only. Its local, greedy/edge, cloud/EDF, and random mean latencies were 3.164, 0.602, 0.513, and 1.177 s, respectively. It is not statistically combined with the Alibaba-derived campaign.

V. CONCLUSION AND PERSPECTIVES

This paper documents a three-tier SimPy architecture and a five-seed, five-noise-seed study of DQN and Double DQN on an Alibaba-derived independent task workload. The evidence is mixed by design: DQN has higher mean reference success in none, periodic, and frequent feedback, whereas Double DQN reduces mean energy in all three modes and reduces P95 in none and frequent modes. Paired intervals include zero, held-out topologies reverse some comparisons, event feedback trades roughly 70% less control traffic for higher P95 at the 0.5 threshold, and fixed SPT beats the learned cloud rule in the reported ablation. These findings support careful scenario analysis, not a universal hierarchical-RL ranking.

Important limitations remain. Alibaba is a datacenter trace adapted with synthetic MEC payloads, deadlines, batteries, links, capacities, and device assignments. The simulator filters multi-task jobs rather than reconstructing DAGs, uses one cloud worker, and models feedback as non-blocking control traffic. The learning formulation is not a full sequential multi-agent Gymnasium environment; cloud next-decision credit and the queue reward are proxies. The campaign artifacts are documented but excluded from Git, so the reported values require artifact release for independent reproduction. Future work should release trace fingerprints and checkpoints, model control-plane contention, add multi-worker cloud resources, evaluate DAGs and calibrated energy, and implement a stepable multi-agent environment. Federated RL remains future work rather than a contribution of this paper.

REFERENCES

[1] ETSI, "Multi-access edge computing (mec); framework and reference architecture," ETSI GS MEC 003 V2.2.1, 2020. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/02.02.01_60/gs_MEC003v020201p.pdf

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys and Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[3] C. Sonmez, A. Ozgovde, and S. Ergen, "Edgecloudsim: An environment for performance evaluation of edge computing systems," in *2017 Second International Conference on Fog and Mobile Edge Computing*, 2017, pp. 39–44.

[4] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "ifogsim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments," *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.

[5] C. Mechalikh, H. Taktak, and F. Moussa, "Pureedgesim: A simulation framework for performance evaluation of cloud, edge and mist computing environments," *Computer Networks*, vol. 182, p. 107474, 2020.

[6] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[7] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[8] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. M. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Transactions on Services Computing*, vol. 17, no. 4, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10409617

[9] H. H. Zhuo, W. Feng, Y. Lin, Q. Xu, and Q. Yang, "Federated deep reinforcement learning," arXiv:1901.08277, 2019. [Online]. Available: https://arxiv.org/abs/1901.08277

[10] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," arXiv:1706.02275, 2017. [Online]. Available: https://arxiv.org/abs/1706.02275

[11] Alibaba Group, "Alibaba cluster trace 2017," ClusterData repository, 2018. [Online]. Available: https://github.com/alibaba/clusterdata/blob/master/cluster-trace-v2017/trace_201708.md